

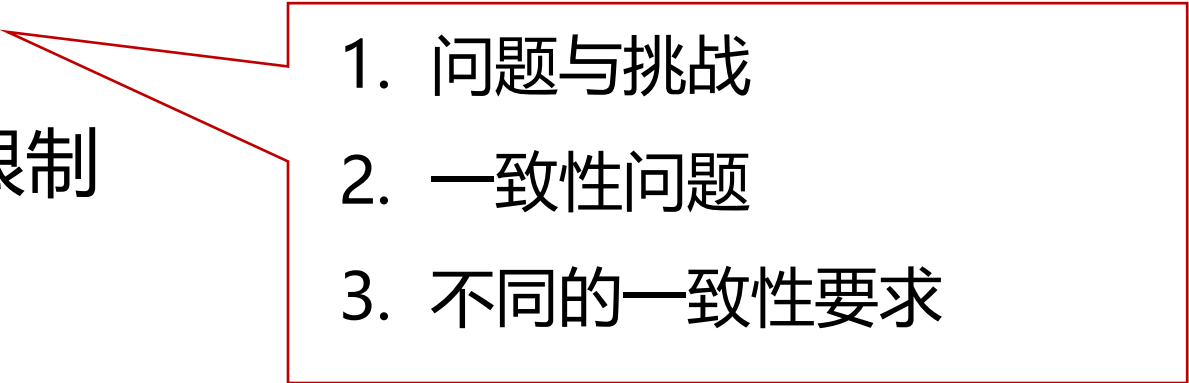
第五章 区块链共识层

本章内容

5.1 一致性问题

5.2 共识设计的理论限制

5.3 区块链共识算法

- 
1. 问题与挑战
 2. 一致性问题
 3. 不同的一致性要求

一致性问题

- 账本的更新需要保障分布式系统的一致性
 - Rivest有100万, Shamir有50万, Adleman有20万
 - TX_RS: Rivest-→Shamir 100w; TX_SA: Shamir-→Adleman 100w
 - A先收到TX_RS后收到TX_SA,A的账本更新为{Rivest: 0, Shamir: 50, Adleman: 120}
 - B先收到TX_SA后收到TX_RS,B的账本更新为{Rivest: 0, Shamir: 150, Adleman: 20}
 - 系统存在两个不一致的账本

问题与挑战

➤ 问题

- 节点之间的网络通信是不可靠的
- 节点的处理时间无法保障
- 节点可以是恶意的
- 同步调用可以简化设计，但会严重降低分布式系统的可扩展性

➤ 可能的解决方案

- 通过同步调用的方法来避免冲突
- 节点提前约好某段时间交易执行顺序的决定权
- 第三方的机构，专门负责处理交易的执行顺序
- ...

一致性要求

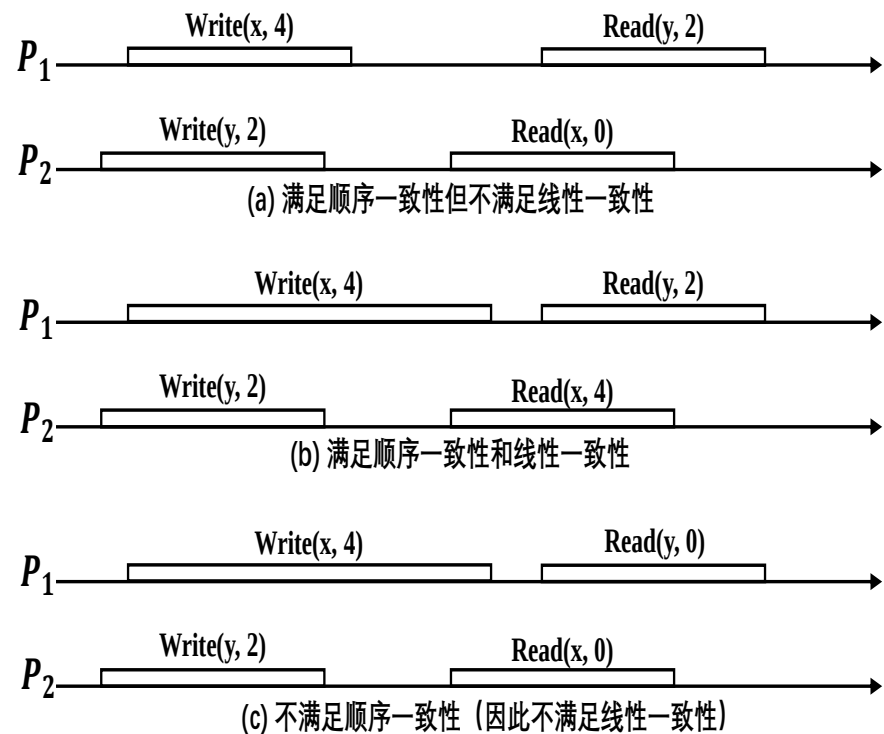
- 分布式系统达成一致性的过程，应该满足
 - 可终止性 (termination)：一致性的结果在有限时间内能完成
 - 约同性 (agreement)：不同节点最终完成决策的结果是相同的
 - 合法性 (validity)：决策的结果必须是某个节点提出的提案
- 对应着三个分布式系统的要求
 - 活性 (liveness)：系统的可用性
 - 安全性 (safety)：任何给出的结果必定是达成共识的
 - 正确性 (correctness)：正确性的共识机制

不同的一致性要求

- 线性一致性 (linearizability consistency) 的要求
 - 任何一次读都能读到某个数据的最近一次写的数据
 - 系统中的所有进程看到的操作顺序，都和全局时钟下的顺序一致
- 顺序一致性 (sequential consistency) 的要求
 - 任何一次读都能读到某个数据的最近一次写的数据
 - 系统中的所有进程看到的操作顺序一致，而且是合理的。即不需要和全局时钟下的顺序一致
- 弱一致性

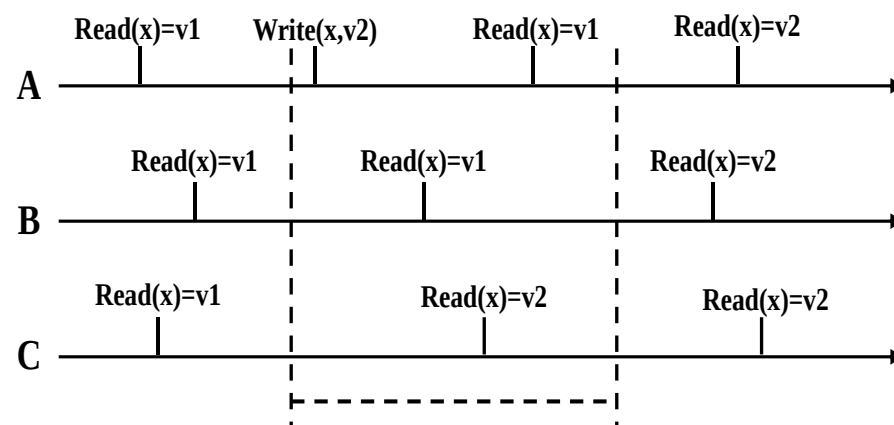
不同的一致性要求

- 线性一致性 (linearizability consistency) 的要求
 - 任何一次读都能读到某个数据的最近一次写的数据
 - 系统中的所有进程看到的操作顺序，都和全局时钟下的顺序一致
- 顺序一致性 (sequential consistency) 的要求
 - 任何一次读都能读到某个数据的最近一次写的数据
 - 系统中的所有进程看到的操作顺序一致，而且是合理的。即不需要和全局时钟下的顺序一致



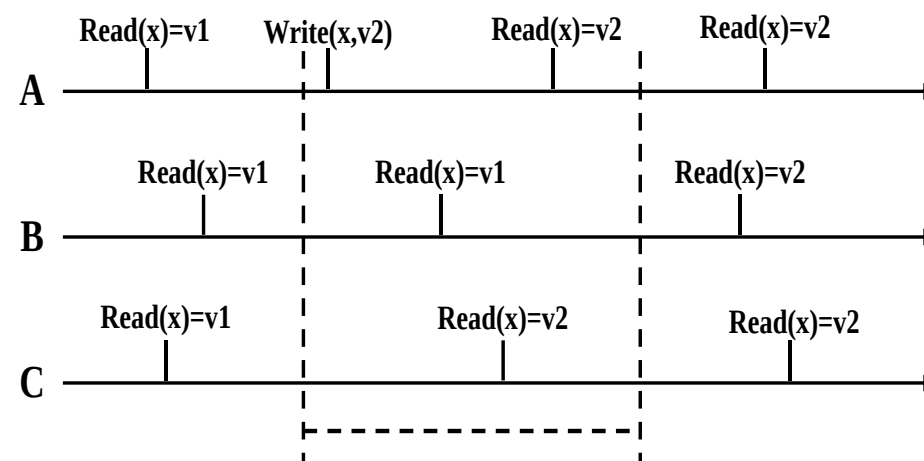
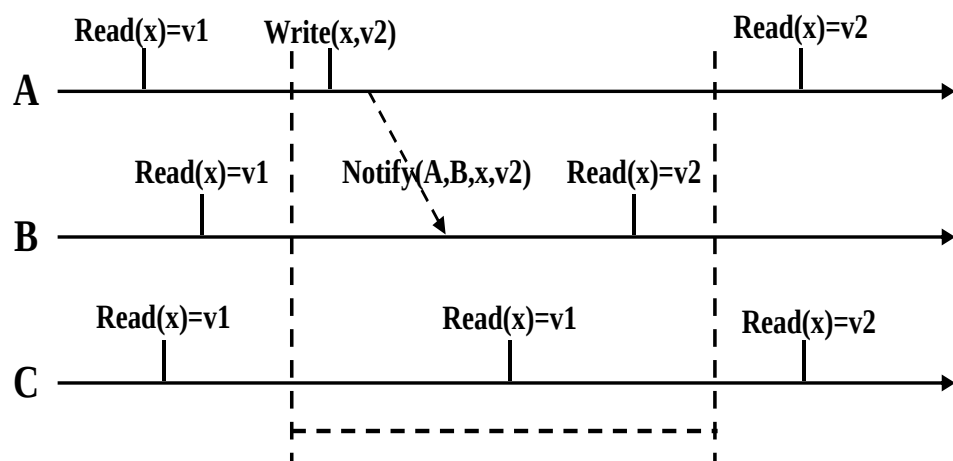
不同的一致性要求

- 最终一致性 (eventual consistency)
 - 放宽对一致性的要求，从而降低系统实现的难度以及复杂性，总会存在某一个时刻（而不是立刻），让系统达到一致的状态
- 根据更新数据后各进程访问到数据的时间和方式的不同
 - 因果一致性 (casual consistency)
 - 读你所写 (read-your-writes)
 - 会话一致性 (session consistency)
 - 单调读一致性 (monotonic read consistency)
 - 单调写一致性 (monotonic write consistency)



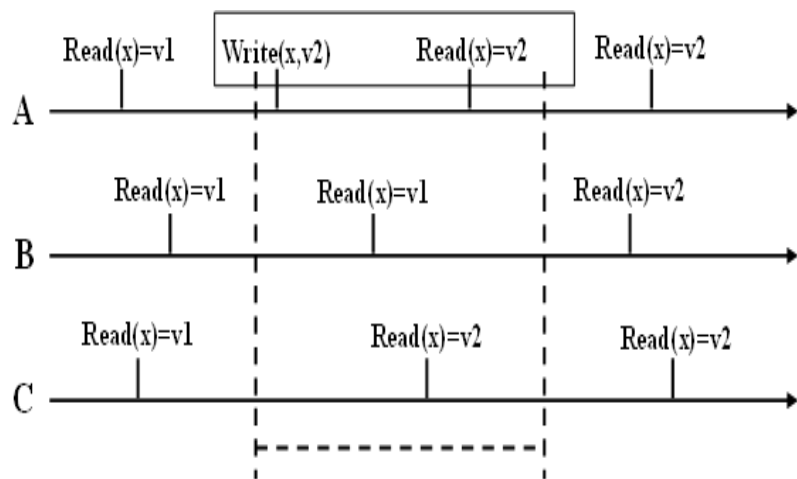
不同的一致性要求

- 因果一致性适用于进程之间有因果依赖的情况
- “读你所写”一致性属于因果一致性范畴中的特例

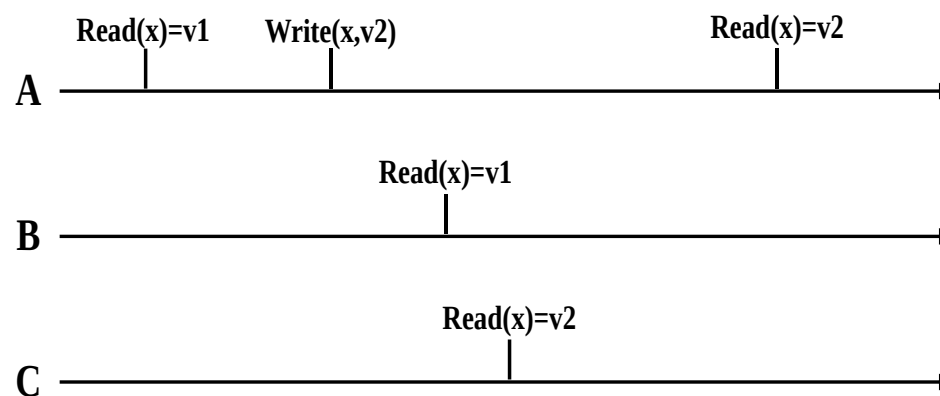


不同的一致性要求

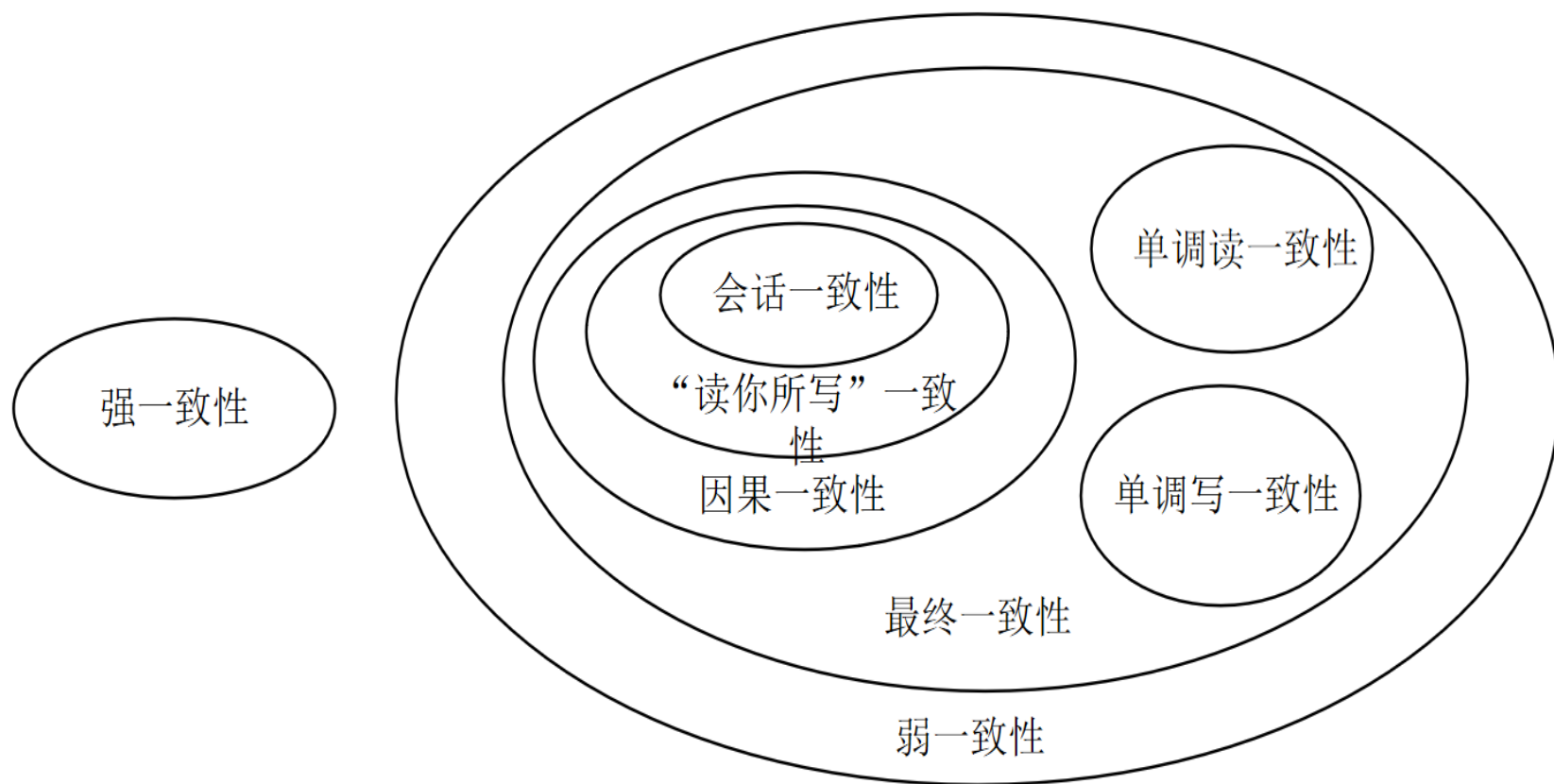
- 会话一致性是“读你所写”一致性的一种变体



- 单调读一致性



不同一致性的关系

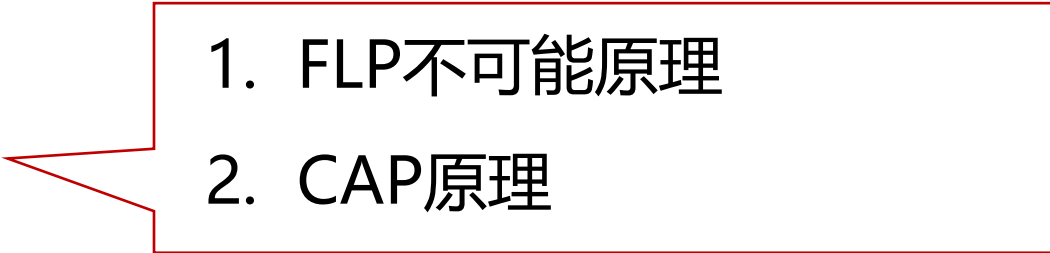


本章内容

5.1 一致性问题

5.2 共识设计的理论限制

5.3 区块链共识算法

- 
1. FLP不可能原理
 2. CAP原理

FLP不可能原理

- 网络通信可靠情况下，分布式系统的共识问题无通用的解法
 - 具体表述：在网络可靠，但允许节点失效（即便只有一个）的最小化异步模型系统中，不存在一个可以解决一致性问题的确定性算法。提出并证明该定理的论文"Impossibility of Distributed Consensus with One Faulty Process"由Michael J. Fischer等科学家于1982年发表
- 启示
 - 不要浪费时间去为一个异步分布式系统设计能在任意情形都能实现共识的确定性算法

FLP不可能原理

➤ 例子

- 四人分别位于全球的四个不同位置对某一个提案进行投票。某个时候，A和D投票同意，而B投票拒绝，C收到三人的投票，然后C睡着了。此时，A、B和D将永远无法在有限时间内获知最终的结果
- 他们无法知道是因为C没有应答（由于时差问题，C可能正在睡觉）还是因为应答的时间过长导致一直无法收到C的答复。
- 如果可以重新投票，则类似情形可以在每次取得结果前发生，这将导致共识过程永远无法完成

➤ 启示

- 对于允许节点失效的情况下，纯粹的异步分布式系统无法保证一致性在有限的时间内完成

FLP不可能原理

- 另一表述：异步的分布式系统不能同时保证活性和安全性
 - 活性反映了来自客户端的请求最终会被处理，即好事总会发生。安全性反映了分布式系统处理了来自客户端的请求后不存在不一致的状态
- 折中方向
 - 弱化活性：PBFT，安全性在异步网络环境中都能得到保证，但可能永远不能完成客户端请求的处理
 - 弱化安全性：工作量证明的活性在异步网络的环境中都能得到保证，但安全性无法保证，可能发生分叉的情况

CAP原理

- 分布式计算系统不可能同时确保以下三个特性
 - 一致性：每次读操作都能得到最近写的结果或者返回错误
 - 可用性：每次请求都返回一个非错误结果，但结果不需要是最近写的结果
 - 分区容忍性：任意节点间的连接中断或大大延迟，系统仍然能够工作
- 如果没有发生网络分区，分布式系统正常地运行，即同时满足可用性以及一致性
 - 比特币更加侧重可用性
 - PBFT更加侧重一致性

本章内容

5.1 一致性问题

5.2 共识设计的理论限制

5.3 区块链共识算法

1. 拜占庭将军问题
2. 拜占庭容错算法
3. 比特币的工作量证明共识机制
4. 以太坊共识算法Ethash
5. 以太坊共识算法Casper (选学)
6. EOSIO共识算法BFT-DPoS (选学)
7. Monoxide (选学)

拜占庭将军问题

- 两将军问题 (Two General Paradox)
 - 两个将军要通过信使来达成进攻还是撤退的约定，但是信使可以被敌军阻拦或者迷路导致消息无法送达（信息丢失或者伪造）
 - 根据FLP不可能原理，两将军问题无通用解
- 拜占庭将军问题 (The Byzantine Generals Problem)
 - Leslie Lamport等科学家于1982年提出用来解释一致性问题的一個虚构模型
 - 将军中可能存在叛徒，如何对军事活动达成一致的决定
 - 将军总数为 N ，叛变将军数为 F ，则当 $N \geq 3F + 1$ 时，问题才有解 (BFT)

拜占庭将军问题

- 拜占庭将军问题 (The Byzantine Generals Problem)
 - 当 $N=3$ 且 $F=1$ 时 ($3 < 3*1+1$) , 即三个将军中存在一个叛徒时, 那么忠诚的将军无法达成行动的一致。主要有以下两种情况:
 - A不是叛徒, 发送一个“进攻”的提案给另外一个忠诚的将军B以及叛徒C; 然而叛变者C可以向将军B宣称自己收到的是“撤退”的提案, 这时候将军B收到两个相反的提案, 无法判断谁是叛徒, 则系统无法达成一致。
 - 另外一种情况, A是叛徒, 分别发送“进攻”和“撤退”的提案给忠诚的将军B和C, 将军B和C都收到两个相反的提案, 无法判断谁是叛徒, 则系统无法达成一致

拜占庭容错算法

- 拜占庭容错算法 (Byzantine Fault Tolerance, BFT)
 - Leslie Lamport等人发表的论文 “Polynomial Algorithms for Byzantine Agreement” 最早提出
 - 复杂度过高, 达到 $O(N^{F+1})$
- 实用拜占庭容错算法 (Practical Byzantine Fault Tolerance, PBFT)
 - 复杂度 $O(N^2)$
 - 在恶意节点不高于总数的1/3并在一个比较弱的同步假设的情况下, 该算法能够同时保证安全性 (Safety) 和活性 (Liveness)
 - 应用于联盟链的场景而不应用于公有链

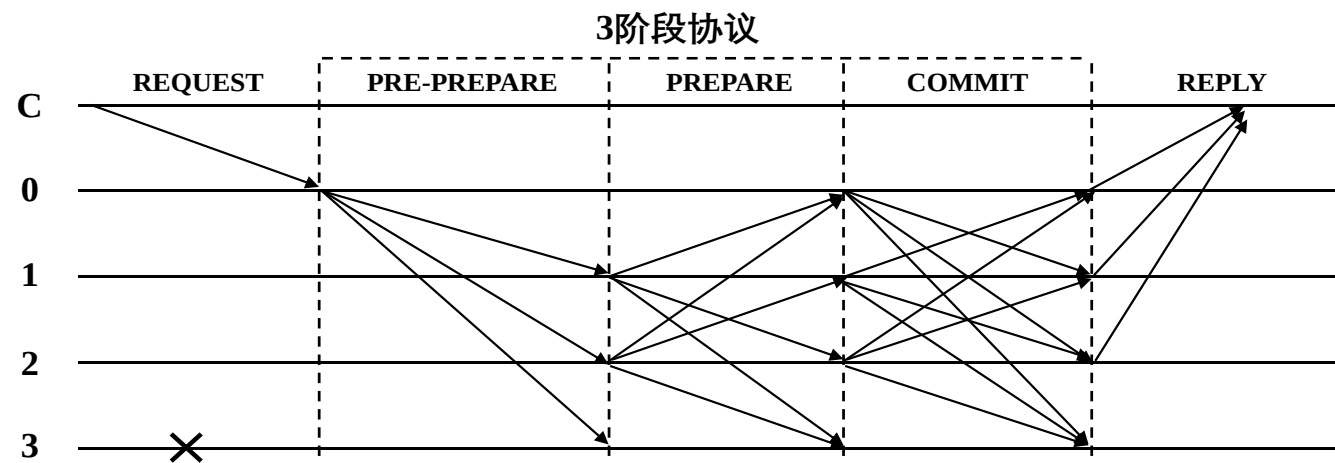
PBFT基本概念

- 客户端 (client) : 向主节点发起请求的客户端, 在区块链中往往跟主节点合二为一
- 主节点 (primary) : 提案发起者, 在区块链中即为区块发起者, 在收到客户端请求后生成新区块并广播
- 验证节点 (backup) : 提案投票者, 在区块链中即为区块验证者, 在收到区块后进行验证, 然后广播验证结果对区块进行投票与共识
- 视图 (view) : 一个主节点和多个备份形成一个视图, 在该视图上对某个提案达成共识, 在区块链中即为对某个区块达成共识。这里需要注意的是, 不同的视图的主节点一般是不一样的, 即主节点是所有节点轮流做的, 每个视图都会重新选择一个主节点
- 编号 (sequence number n) : 在每个视图中由主节点指定的一个数字, 即提案的编号, 在区块链中可以理解为区块高度
- 检查点 (checkpoint) : 如果某个编号 n 对应的提案 (区块) 收到了超过 $2/3$ 的确认, 则称为一个检查点

PBFT具体流程

➤ 三个阶段

- 预准备阶段（PRE-PREPARE阶段），准备阶段（PREPARE阶段），提交阶段（COMMIT阶段）
- 图中的C为客户端，0,1,2,3为共识节点，其中共识节点0-2正常执行，而共识节点3为故障节点，表现为对其它节点的请求无响应



PBFT具体流程

➤ 预准备阶段

- 主节点收到客户端发来的请求后构造PRE-PREPARE消息结构体 $\langle\langle \text{PRE-PREPARE}, v, n, d \rangle, m \rangle$ ，签名广播给其他节点
- 验证节点收到主节点的PRE-PREPARE消息后，会对消息进行检验
- 检验通过之后，节点会构造REPARE消息结构体 $\langle\langle \text{PREPARE}, v, n, d, i \rangle \rangle$ ，签名后广播给其他节点， i 为本节点的编号
- 在此过程节点将PREPARE消息和PRE-PREPARE消息记录到本地的日志中，用于视图切换中恢复未完成的请求，并重播到网络中在新的视图中继续共识

PBFT具体流程

➤ 准备阶段

- 当节点收到其他节点的PREPARE消息，会进行如下检验：检验PREPARE消息的签名是否正确；当前节点是否已经收到与PREPARE消息中 v, n 对应的PRE-PREPARE消息，且 n 在区间 $[h, H]$ 之间； d 和当前已收到PRE-PREPARE消息中的 d 是否一致
- 当节点在规定时间内收到 $2F+1$ （包括自己）个通过检验的PREPARE消息后，则会构造COMMIT消息结构体 $\langle \langle \text{COMMIT}, v, n, d, i \rangle \rangle$ ，签名广播给其他节点
- 在此过程中节点将COMMIT消息和其他节点发送的PREPARE消息记录到本地的日志中，用于视图切换中恢复未完成的请求。主节点发送的消息已经达到PREPARED状态

PBFT具体流程

➤ 提交阶段

- 当节点收到来自其他节点的COMMIT消息时，同样会对COMMIT消息进行检验，步骤与对PREPARE消息的相似
- 节点在规定时间内收到 $2F+1$ 个（包括自己）通过检验的COMMIT消息，说明当前网络中的大部分节点已经对当前视图内的请求达成共识
- 节点会处理消息 m 中包含的操作。如果有多个 m ，则按照序号 n 从小到大执行。执行完成后，节点会返回结果给客户端。同样，在该过程，节点会将其他节点发送的COMMIT消息记录到本地的日志中
- 当客户端收到 $F+1$ 个相同的结果（来自不同节点）时，说明已经在全网达成共识，否则客户端需要判断是否重新向主节点发起请求

日志压缩

➤ 理想情况

- 每执行K个请求后，节点i会创建一个检查点并广播消息 $\langle \text{CHECKPOINT}, n, d, i \rangle$ 给网络中的其他节点，其中i为本节点的编号，n是最后一次执行的请求的序号，d为执行序号为n的请求后的状态机的状态摘要
- 此外该检查点消息同样会被记录到本地的日志中。当节点i收到 $2F+1$ 个（包括自己）通过检验的检查点消息以后，则清除序号小于n的消息。同时，该检查点变成稳定检查点，其高度为n

日志压缩

➤ 实际情况

- 当节点i向其他节点发出检查点消息后，其他节点并未处理完K个请求，因此并不会立刻对节点i作出响应
- 节点i并不会等待其他节点的回复，而是继续按照自己的节奏继续向前处理请求，但是节点i创建的检查点在此时并未变成稳定检查点
- 为了防止节点i处理请求过快，导致其创建的多个检查点一直无法变成稳定检查点，从而使得日志积压
- PBFT设置了上述提及的高低水位区间 $[h, H]$ 。其中低水位 h 等于上一个稳定检查点的高度，高水位 $H = h + L$ ，其中 L 是系统设定的数值，等于节点周期处理请求数值 K 的整数倍
- 例如可以设置 $L = 3K$ 。当节点i处理的请求序号到达高水位 $H = L + 3K$ ，便会暂时停止脚步，直到稳点检查点发生变化再继续向前，从而避免日志的积压

视图切换

- 在每个视图开始或者节点收到消息时，节点i会启动一个计时器，如果此时刚好有定时器在运行，则会重置计时器
- 但当主节点宕机或者其他情况导致节点i在当前视图v超时的时候，节点i就会触发视图切换的操作，启动视图切换的协议
- 节点i向其他节点广播<VIEW-CHANGE,v+1,n,C,P,i>消息
- 其中n是最新的稳定检查点的高度，C是节点i保存的经过 $2F+1$ 个（包括自己）节点确认稳定检查点消息的集合，表明该稳定检查点已经达到全网共识，P是当前节点保存的n之后所有已经达到PREPARED状态消息的集合，即已完成准备阶段的消息

视图切换

- 当新视图 $v+1$ 中的主节点 i 收到 $2F+1$ 个（包括自己）经过检验的VIEW-CHANGE消息，则向其他节点广播 $\langle \text{NEW-VIEW}, v+1, V, O \rangle$ 消息， V 是有效的VIEW-CHANGE消息集合， O 是pre-prepare消息集合
- 该集合是从已经达到PREPARED状态的消息中转换过来的。当其他验证节点收到主节点的NEW-VIEW消息后，校验签名， V 和 O 的消息是否合法，验证通过后，则主节点和验证节点都进入新视图
- C 确保了视图切换时，在最新稳定点之前的状态安全， P 和 O 确保了视图切换中达到PREPARED状态的消息能够得到重放，而不会丢失
- 预准备阶段和准备阶段是为了确保同一视图下消息的一致性，而 C ， P 和 O 三个集合就是解决这个问题的，确保在新视图中，主节点重放的旧视图中的消息依旧能保持顺序一致性

PBFT中节点的状态

- 等待请求：等待客户端的请求，所有节点均处于这个状态，主节点收到客户端请求之后，会切换到预准备状态，而验证节点收到主节点消息后会切换到准备状态。
- 预准备：这个状态是主节点专属的，主节点处理客户端的请求并广播PRE-PREPARE消息，切换到准备状态
- 准备：节点进入这个状态后，验证主节点的PRE-PREPARE消息并生成和广播PREPARE消息，同时等待 $2F+1$ 个（包括自己）节点的PREPARE确认
- 等待 $2F+1$ 个PREPARE确认：如果在超时之前收到 $2F+1$ 个PREPARE确认，则切换到提交状态；否则，则切换到视图切换状态
- 视图切换：节点广播编号为 $v+1$ 的VIEW-CHANGE消息，并等待 $2F+1$ 个（包括自己）VIEW-CHANGE消息
- 等待 $2F+1$ 个VIEW-CHANGE消息

PBFT的一些思考

➤ 为什么PBFT的容错性是1/3?

- 拜占庭节点可以故意不回复，因此节点必须在收到 $N-F$ 个回复后就要作出决策，节点收到的 $N-F$ 个回复有可能包含 F 个拜占庭节点的回复，那么正确的消息就是 $N-F-F$ 个，为了遵循少数服从多数的原则，让诚实节点能达成共识，必须使得 $N-F-F > F$ ，因此， $N > 3F$

➤ 为什么需要提交阶段?

- 一是因为节点A收到 $2F+1$ 个PREPARE消息并不代表其他节点也收到足够的 $2F+1$ 个PREPARE消息。想象一下，当节点A收到 $2F+1$ 个PREPARE消息便执行请求，甚至返回结果给客户端。这时，部分节点B发生了视图切换，你认为达成共识的请求其实没有达成共识，而是需要在新视图重播的，因此节点A返回结果给客户端是不合理的

PBFT的一些思考

➤ 为什么需要提交阶段？

- 二是因为在节点B尚未收到足够的PREPARE消息，在视图切换的过程中也是拒收PREPARE消息的，因此这部分节点的状态机会少执行一个请求。当视图切换后，该请求对应的PREPARE消息会被重放，而节点A收到重放的PREPARE消息会面临该请求已经被执行过，是否需要再次执行，这将导致状态机的二义性

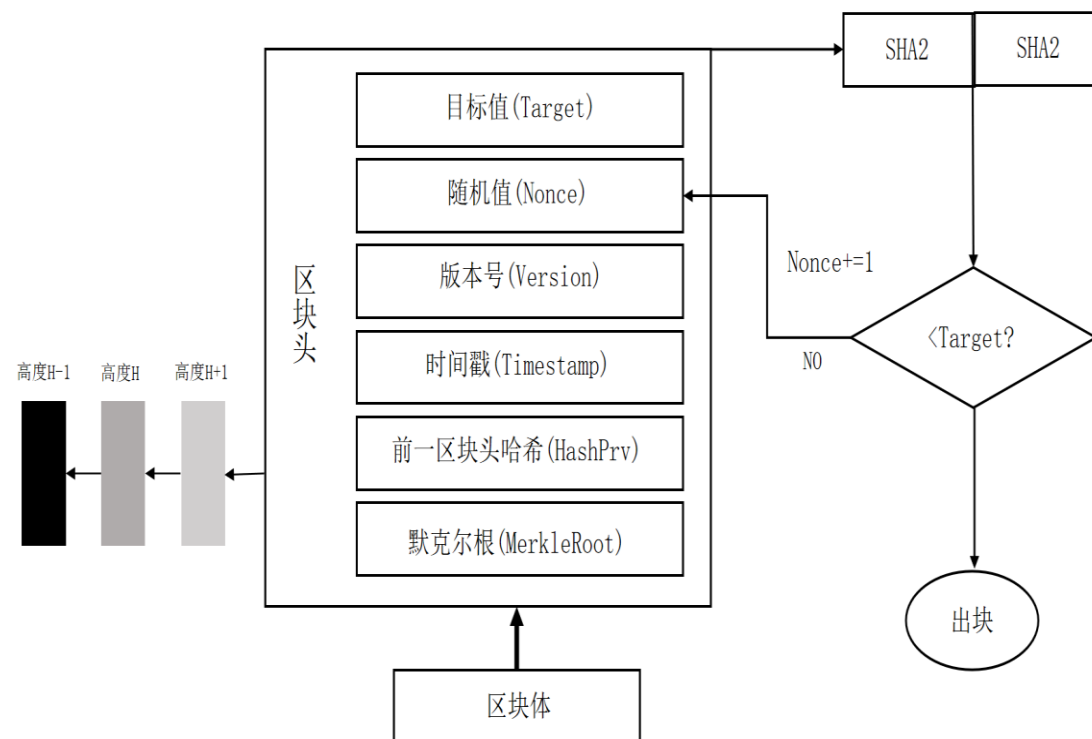
➤ PBFT的复杂度是多少？

- 预准备阶段和准备阶段，网络中的每一个节点都需要给其他节点广播消息，假设总节点数为 N ，则通讯开销的复杂度为 $N*(N-1)$ ，因此PBFT的（通信）复杂度为 $O(N^2)$

比特币的工作量证明共识机制

➤ 比特币工作量证明过程

- 矿工对区块头的随机值字段Nonce进行遍历。每次遍历都对区块头进行两轮SHA256哈希算法（隶属SHA2）。只有当区块头两轮哈希后得到的值小于工作量目标值，矿工才算完成了工作量证明
- 两轮哈希算法的设计主要是为了避免哈希算法SHA256潜在的延展性攻击。哈希算法的单向性及映射结果的均匀性保证了矿工不能从工作量目标值反向推出合法随机值



比特币的工作量证明共识机制

➤ 比特币工作量证明过程

- 某矿工率先完成工作量证明，则该矿工广播该区块至区块链网络。区块链网络中的其他矿工把接收到的区块作为候选区块进行验证
- 若验证通过，则把该区块插入本地的数据库中并进入下一轮共识
- 若验证不通过，则继续进行工作量证明
- 算力占优的诚实矿工会一直基于最长链进行共识并延伸最长链
- 比特币选择最长链作为全局账本的原因是最长链承载着次数最多的工作量证明

比特币的工作量证明共识机制

➤ 难度控制

- 通过控制哈希得到的值的大小（前方0的个数）
- 当网络算力大（全网每秒的哈希次数多时），需要提升全网难度。当网络算力小，需要降低全网难度

➤ 双花攻击

- 攻击者付钱购买商品，然后商家等待该交易所在的区块N被后续足够多的区块所引用，则给攻击者发货。攻击者在付钱的同时，开始在暗地里用自己的算力来支持双花交易（可以使付钱交易无效的交易），即从高度N开始制造分叉
- 若攻击者的算力足够大，则暗地里的分叉链的长度会大于当前主网的链。当攻击者收到货以后，就会广播分叉链，由于分叉链的长度比当前主网链要长，所以区块链网络舍弃当前主网链，采用分叉链为最新主网链，从而付费交易失效，而双花交易成功

比特币的工作量证明共识机制

- 诚实矿工的算力占全网算力的比例为 p 而攻击者的算力占全网的比例为 q ，其中 $p+q=1$
- 攻击者在落后 z 个区块的情况下，攻击者能够追上诚实矿工的概率下述公式，其中 $\xi=q/p$

$$\text{➤ } Q_{\xi}(z) = \begin{cases} \xi^z & \xi < 1 \text{ 且 } z > 0 \\ 1 & \xi \geq 1 \text{ 或 } z \leq 0 \end{cases}$$

- y 确认代表当交易被 y 个区块所支持时交易被认为几乎不可能被逆转。比特币中的 y 常被取为6。而当主链上观察到 y 确认时，攻击者暗地挖到的区块数量 x 满足参数为 $\lambda=\xi y$ 的泊松分布，并记为 $p(x)$ 。攻击者攻击成功的概率见下述公式
- $P(\text{攻击成功}; y\text{确认}) = \sum_{x=0}^{+\infty} p(x) Q_{\xi}(y-x) = 1 - \sum_{x=0}^y \frac{\lambda^x e^{-\lambda}}{x!} (1 - (\frac{q}{p})^{y-x})$
- 随着 y 的增大， $p(\text{攻击成功}; y\text{确认})$ 呈现指数级别下降

比特币区块头的六个主要字段

字段	存储大小	描述
Version	32 bits	区块的版本信息
hashPrevBlock	256 bits	上一个区块的哈希
hashMerkleRoot	256 bits	比特币交易构造的默克尔树的根
Timestamp	32 bits	区块的时间戳（秒级别）
Target	32 bits	当前的区块目标值
Nonce	32 bits	用于工作量证明生成合法区块

```
import hashlib
# 分别对应上述的6个小端模式字段
header_hex = ("01000000" +
"81cd02ab7e569e8bcd9317e2fe99f2de44d49ab2b8851ba4a308000000000000" +
"e320b6c2fffc8d750423db8b1eb942ae710e951ed797f7affc8892b0f1fc122b" + "c7f5d74d" + "f2b9441a" + "42a14695")
# 把string解码成hex的形式
header_bin = header_hex.decode('hex')
# 双重hash
hash = hashlib.sha256(hashlib.sha256(header_bin).digest()).digest()
# hash的小端模式结果，比特币区块中存的是这个结果
hash.encode('hex_codec')
'1dbd981fe6985776b644b173a4d0385ddc1aa2a829688d1e00000000000000000'
# hash的大端模式结果
hash[::-1].encode('hex_codec')
'000000000000000001e8d629a8a21adc5d38d0a473b144b6765798e61f98bd1d'
```

以太坊共识算法Ethash

➤ 算法摘要

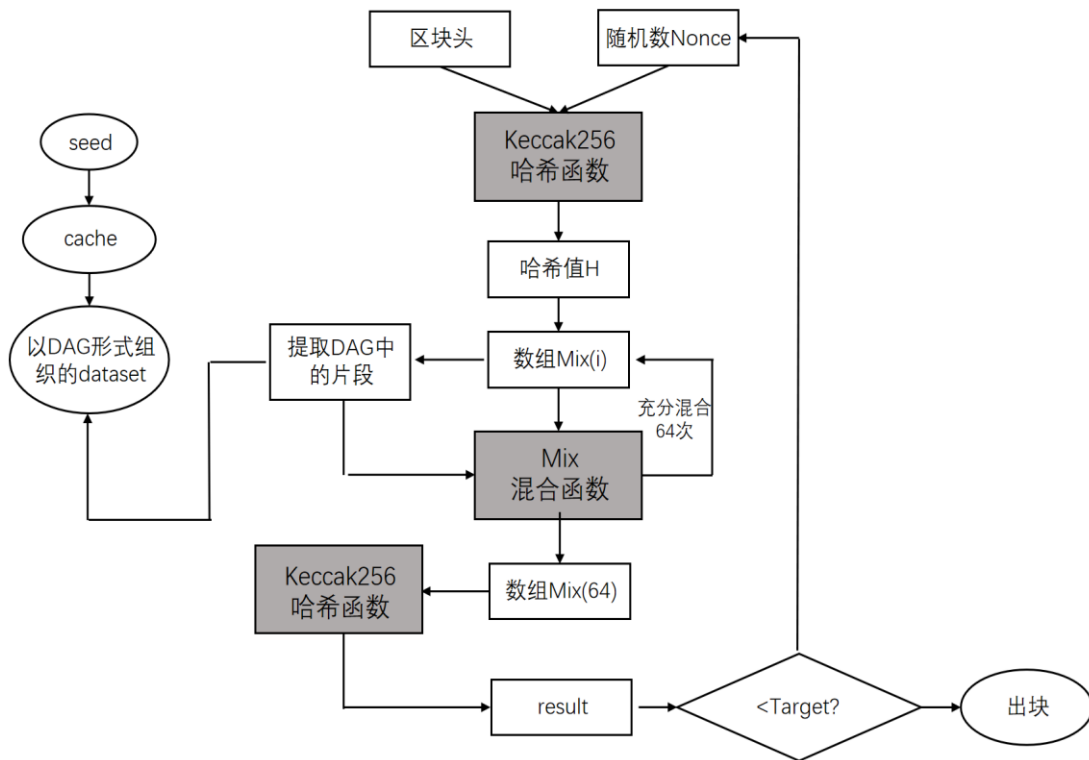
- 跟比特币的矿工不同，以太坊中的矿工在挖矿过程中不仅需要找到一个合适的Nonce填充到区块头中，还需填充一个MixDigest字段，这是矿工在挖矿过程中计算出来的，作为矿工挖矿过程中的内存消耗证明。在矿工验证区块的时候也会用到MixDigest字段
- 在挖矿前，矿工在挖矿会根据区块高度生成一个种子seed，然后根据seed生成一个约16MB的缓存cache，这些cache由轻节点存储
- 全节点会根据cache用特定的算法生成一个GB级别的dataset，dataset中每一项都是由cache中的一小部分数据计算出来的，其大小随着时间增长。这是为了抵消摩尔定律下硬件性能的提升。此外，dataset可以理解为一个以有向无环图（DAG）形式组织起来的随机数序列，后续在挖矿过程中被利用好
- cache和dataset的内容并非不变，而是每隔一个纪元epoch（30000个区块）由新的seed重新计算。新的seed的取值仅与区块高度相关，因此dataset是可以预生成的。如果没有预生成，节点需要等待dataset生成才能继续挖矿出块操作，这将导致在每个纪元的过渡时间，网络也会经历一个较大的区块延迟。（注：当首次启动一个节点时，挖矿公示只会在dataset生成之后才开始）
- 挖矿工作包含了抓取dataset的随机片段以及运用哈希算法计算它们。验证工作能够在低内存的环境下进行，通过使用cache来生成所需的特定片段，所以轻节点主要存储cache便可以完成验证工作

以太坊共识算法Ethash

➤ 挖矿流程

- 首先依据区块头和填充的随机值通过Keccak256哈希算法（隶属SHA3）生成哈希值H，并将H整合为一个包含32个uint32元素的数组Mix(0)（一共128个字节）
- 将Mix(0)通过某个特定算法映射到dataset中，抓取dataset某个片段数据data
- 将data和Mix(0)通过FNV哈希算法进行混合，得到新的混合结果Mix(1)
- 基于Mix(1)再去抓取dataset中某个片段，继续上述的混合操作，循环混合64次之后得到最后的混合结果Mix(64)
- 将Mix(64)整合为32字节的digest，并依据H和digest采用Keccak256哈希算法生成最终的哈希值result，并与目标值进行比对，判断其是否满足出块的条件
- 若满足条件，则将digest和Nonce分别填充到区块头的MixDigest字段和Nonce字段，其他节点将利用这两个字段对该区块进行工作量证明的相关验证

```
func fnv(a, b uint32) uint32 {  
    return a*0x01000193^b  
}  
  
func fnvHash(mix []uint32, data []uint32) {  
    for i := 0; i<len(mix); i++ {  
        mix[i]=mix[i]*0x01000193^data[i]  
    }  
}
```



以太坊共识算法Ethash

➤ 特点

- 需要GB级别的dataset不断地读取片段数据
- 相比于比特币工作量证明，Ethash增加了对内存消耗的要求，要求矿工提供在挖矿过程中使用大量内存的证明，最终达到抵抗ASIC的目的
- 支持轻节点对区块进行验证，验证节点只需保存一个16MB的cache
- Ethash侧重的是工作量证明过程中快速IO的能力而比特币工作量证明侧重的是哈希速率，均在做无意义的哈希或IO

以太坊共识算法Casper（选学）

➤ 权益证明

- 公有区块链采用共识机制主要有两种：工作量证明（PoW）和权益证明（PoS）
- PoW能源消耗巨大，PoS基于某种稀缺资源对记账权进行竞争
- 权益证明中的稀缺资源为权益，权益的表现形式大多为公有区块链上的加密货币，包括持币量以及币龄等
- 主要面临两个挑战：长程攻击（long range attack）和无利害关系（nothing at stake）

以太坊共识算法Casper（选学）

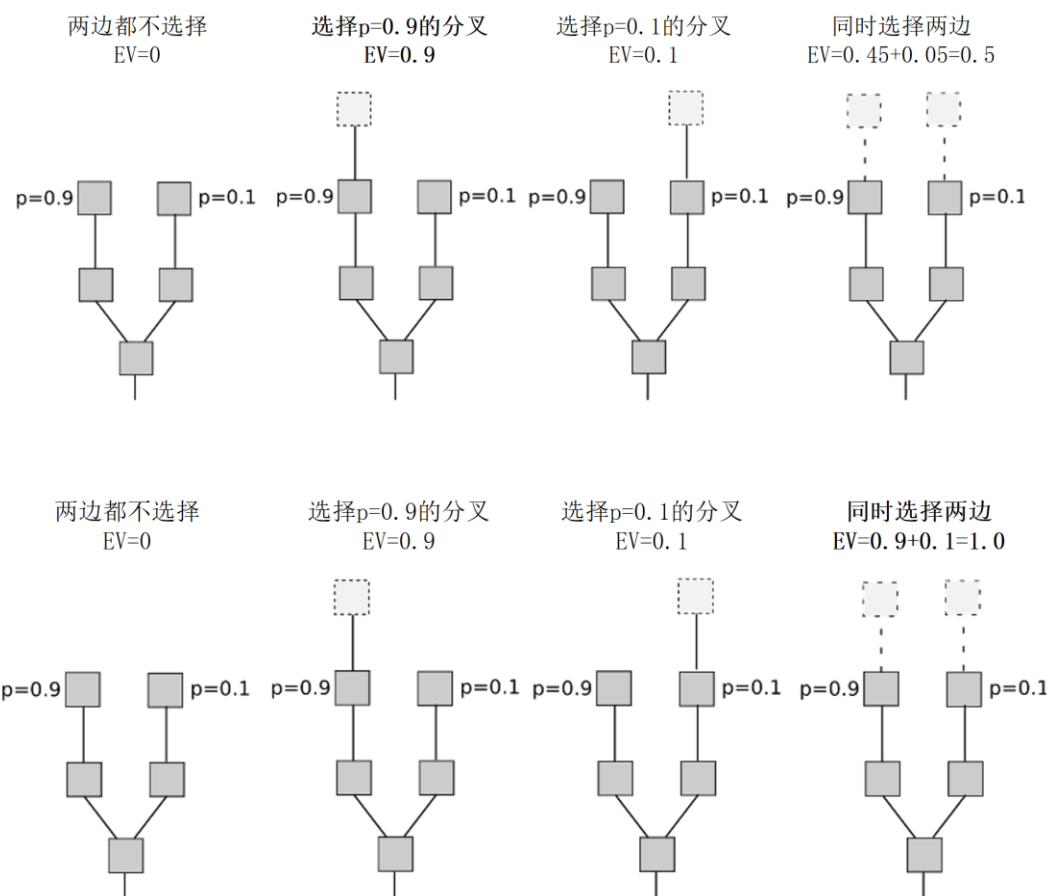
➤ 长程攻击

- 区块链矿工/验证者在退回抵押的权益后，从历史上的某个区块开始重写后续区块，由于矿工/验证者已经没有保证金，系统就不能对该攻击矿工/验证者进行惩罚
- 新参与者需要咨询受信任节点以安全地冷启动，该方式被称为“主观依赖”（Weak Subjectivity）。当前该问题只能缓解而不能解决，可以被称为权益证明的达摩克利斯之剑
- 工作量证明不会出现这个问题

以太坊共识算法Casper (选学)

➤ 无利害关系

- 相对于工作量证明矿工的每份算力只能选择一个分叉进行投入, 权益证明矿工的每份权益可以同时所有的分叉上押宝
- 导致权益证明区块链长时间维持分叉的状态



以太坊共识算法Casper（选学）

➤ Casper FFG

- 拜占庭容错（BFT）风格的权益证明，其目前容错率为 $1/3$
- 针对长程攻击，Casper FFG要求验证者在离开验证者集合拿回抵押的保证金前必须要经过一个解冻期以符合同步的前提，比如四个月
- 只算缓解了长程攻击的问题，并不能解决
- 保证金的引入解决了无利害关系的问题：同时支持两个互不兼容的分叉时，系统会对恶意投票者的保证金进行划扣

EOSIO共识算法BFT-DPoS（选学）

➤ 委托权益证明（Delegated Proof of Stake, DPoS）

- 委托权益证明在成千上万个PoS节点中，通过某种机制（例如持有代币的数量）选举出若干（奇数个）代表节点，进而在这几个代表节点之间进行投票选举出新区块的创建者
- 大幅度提升选举效率

➤ BFT-DPoS

- 为了缩短传统的DPoS的交易确认时间（45秒），EOSIO借鉴PBFT共识机制的思想进行改进，从 45 秒缩短至3 秒左右
- 为了挖掘更高的性能，EOSIO在初级版BFT-DPoS共识机制的基础上又进行了修改，将出块速度从之前的3秒缩短至0.5秒

Monoxide (选学)

➤ Monoxide

- "Monoxide: Scale Out Blockchain with Asynchronized Consensus Zones", 2019年由计算机网络顶级学术会议 NSDI所接收
- 利用分片的思想把区块链划分为多个共识组以提升区块链的可扩展性
- 单一系统分割成一定程度上独立的N个共识组子系统, 以线性地提升整个系统的交易处理速度与吞吐量
- 交易的吞吐量及TPS约为原来的N倍
- 在连弩挖矿这种模式下, 矿工算力不会稀释到多个共识组上, 使得攻击者集中算力去攻击单个共识组也不能获得优势
- 连弩挖矿会带来潜在的中心化问题

本章总结

- 一致性问题与挑战
- 不同的一致性要求
- FLP不可能原理
- CAP原理
- 拜占庭将军问题
- 拜占庭容错算法
- 比特币的工作量证明共识机制
- 以太坊共识算法Ethash
- 以太坊共识算法Casper、EOSIO共识算法BFT-DPoS、Monoxide（选学）